

Automated Medical Podcast Transcription and Topic Segmentation

1. Introduction

In recent years, medical podcasts have emerged as a valuable source of information for medical students, healthcare professionals, researchers, and educators. These podcasts cover a wide range of topics including diseases, symptoms, treatment methods, clinical experiences, case discussions, and public health awareness. Due to their conversational and explanatory nature, podcasts help listeners gain deeper insights into real-world medical scenarios.

Medical podcasts are also increasingly used as supplementary learning resources alongside textbooks and journals. Experts often share real-life experiences and updated medical practices, making podcasts an effective medium for continuous medical education.

However, most medical podcasts are lengthy and unstructured. Identifying a specific disease discussion or treatment explanation within a long audio episode requires listening to the entire podcast, which is both time-consuming and inefficient. This creates a major challenge for users who wish to quickly access relevant medical information.

To address this issue, there is a strong need for an intelligent AI-based system that can automatically convert medical podcast audio into text, divide it into meaningful topic-based segments, and provide structured insights such as keywords, sentiment analysis, and transcription quality evaluation.

The **Automated Medical Podcast Transcription and Topic Segmentation** project aims to solve this problem using Artificial Intelligence (AI) and Natural Language Processing (NLP) techniques. The system transforms raw medical audio into searchable, structured, and user-friendly content, enabling efficient access and analysis of medical podcast information.

2. Problem Statement

Medical podcasts contain highly valuable educational and clinical information, but their long duration and lack of structure make them difficult to analyze and navigate. Users face several challenges, such as locating specific disease or treatment discussions, understanding topic transitions, and evaluating the quality and reliability of the content.

Since podcasts are audio-based, traditional search and indexing methods cannot be applied directly. This makes it difficult for users to reuse podcast content for academic or reference purposes.

Currently, there is no effective automated mechanism that can process medical podcast audio, extract meaningful topics, and provide analytical insights in a structured format. As a result, users are forced to rely on manual listening and interpretation.

The problem addressed by this project is the design and development of an intelligent system that can automatically transcribe medical podcast audio, detect topic boundaries within the transcript, extract keywords, analyze sentiment, evaluate transcription quality, and present the results through an interactive user interface.

3. Objectives of the Project

The primary objective of this project is to develop an AI-driven system that improves the accessibility and usability of medical podcast content. The system aims to accurately convert medical audio into text using advanced speech-to-text models and organize the information in a structured manner.

The project also aims to segment long transcripts into meaningful medical topics, making it easier for users to navigate large podcast episodes. Another objective is to extract important medical keywords that summarize each topic segment.

In addition, the system performs sentiment analysis in a medical-aware manner and evaluates transcription quality using metrics such as Word Error Rate (WER), Character Error Rate (CER), and semantic similarity. The final objective is to provide an interactive frontend interface for visualization and analysis.

4. Scope of the Project

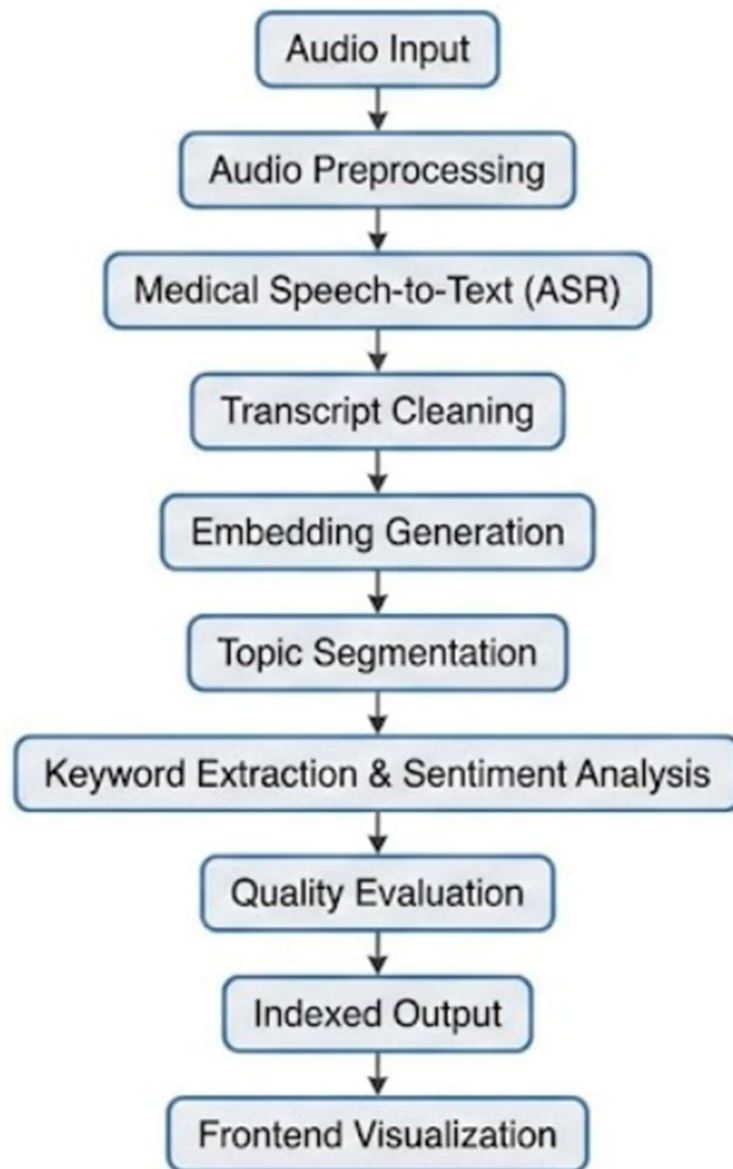
The scope of this project includes the processing of recorded medical podcast audio files. It focuses on automatic transcription using Automatic Speech Recognition (ASR) models, topic segmentation using NLP techniques, keyword extraction, sentiment analysis, and transcription quality evaluation using standard metrics.

The project also includes visualization of results through a web-based interface for better user interaction.

The project does not include live audio streaming or real-time transcription. It also does not provide clinical diagnosis or treatment recommendations, as the system is intended strictly for educational and informational purposes.

5. System Architecture :

The system uses a pipeline-based modular architecture to process medical podcast audio. The audio is transcribed using speech-to-text models and analyzed using NLP techniques for topic segmentation, keyword extraction, sentiment analysis, and quality evaluation. The results are stored and visualized through an interactive frontend interface.



6. Module Description

6.1 Audio Preprocessing Module

The Audio Preprocessing Module is the initial stage of the system pipeline and is responsible for converting raw medical podcast audio into a standardized format suitable for further processing. Medical podcast recordings often vary in quality due to differences in recording environments, microphone quality, background noise, speaker distance, and volume inconsistencies. If these issues are not addressed, they can significantly affect the accuracy of speech recognition.

This module applies a sequence of audio signal processing operations to enhance speech clarity and consistency. All input audio files are first converted into a uniform format with a fixed sampling rate and mono channel configuration to ensure compatibility with speech-to-text models. Noise reduction techniques are then applied to suppress background disturbances such as ambient noise and static interference. Silence detection algorithms identify and remove non-speech segments to reduce unnecessary processing. Volume normalization is performed to maintain consistent audio levels throughout the recording. For long podcast episodes, the audio may be divided into smaller chunks to ensure efficient processing and stable system performance.

6.2 Transcription Module

The Transcription Module converts the preprocessed medical podcast audio into textual form using Automatic Speech Recognition (ASR) techniques. This module is designed to handle long-duration conversational speech while preserving linguistic structure, context, and meaning. Medical podcasts often contain complex terminology, abbreviations, and clinical references, which require robust transcription models.

Advanced ASR models such as Whisper or Faster-Whisper are used to generate accurate transcriptions. These models analyze acoustic signals and linguistic patterns to produce text that closely reflects the spoken content. During transcription, the audio is segmented into time-aligned units, and timestamps are generated for each segment. This alignment enables precise synchronization between the audio and the transcript. The generated transcription is stored in a structured format, enabling further analysis by natural language processing modules.

6.3 Topic Segmentation Module

The Topic Segmentation Module organizes lengthy medical podcast transcripts into coherent and meaningful topic-based sections. Medical discussions often shift naturally between topics such as symptoms, diagnosis, treatment options, clinical experiences, and public health information without explicit indicators.

To identify these transitions, the transcript is first divided into sentences. Each sentence is converted into a semantic vector representation using sentence-level embeddings. Similarity scores are computed between consecutive sentence embeddings to detect changes in semantic context. When a significant drop in similarity is observed, the system identifies a topic boundary. Based on these detected boundaries, the transcript is segmented into topic-specific sections that reflect logical transitions in the discussion.

6.4 Keyword Extraction Module

The Keyword Extraction Module identifies important medical terms and phrases that characterize each topic segment. Medical podcast transcripts often contain detailed explanations, making it necessary to extract representative keywords for quick understanding and indexing.

Text from each topic segment undergoes preprocessing steps such as tokenization and stop-word removal. Contextual and statistical keyword extraction techniques, including KeyBERT, YAKE, and RAKE, are applied to analyze term relevance, phrase structure, and contextual importance. The extracted keywords provide a concise summary of the medical concepts discussed in each segment and support efficient navigation and thematic analysis.

6.5 Sentiment and Quality Evaluation Module

This module performs analytical evaluation of both the emotional tone and transcription accuracy of the medical podcast content. Sentiment analysis is carried out at the topic-segment level to capture contextual meaning across discussions. Since medical conversations often involve serious or sensitive topics, the sentiment analysis process is adapted to ensure that factual and clinical explanations are not incorrectly classified as negative.

In addition to sentiment analysis, transcription quality evaluation is performed to assess the reliability of the generated transcripts. When reference transcripts are available, the system compares them with the generated output using standard evaluation metrics such as Word Error Rate (WER) and Character Error Rate (CER). Semantic similarity analysis is also conducted to evaluate how accurately the meaning of the spoken content is preserved in the transcription.

6.6 Frontend Visualization Module

The Frontend Visualization Module presents the processed medical podcast information in a structured and interactive format. It serves as the user-facing component of the system and enables intuitive exploration of transcripts and analytical results.

The interface displays complete transcripts along with topic-based segmentation, extracted keywords, sentiment indicators, and transcription quality metrics. Timestamp-based navigation allows users to directly access specific sections of the original podcast audio. Interactive visual components such as dashboards and graphs support efficient analysis and improve usability. The frontend communicates with the backend through RESTful APIs, ensuring seamless data exchange and real-time visualization of results.

7. Usage of the System

This section describes the complete procedure for setting up, executing, and using the Automated Medical Podcast Transcription and Topic Segmentation system.

7.1 System Requirements

To run the system locally, the following requirements must be met:

- Python 3.9 or higher
- Git version control system
- FFmpeg (for audio preprocessing)
- Node.js 16+ (optional, for frontend UI)

7.2 Project Setup and Installation

Clone the project repository and navigate to the project directory:

```
git clone https://github.com/springboardmentor13579x-proj/Automated-Podcast-Transcription-and-Topic-Segmentation.git
```

```
cd Automated-Podcast-Transcription-and-Topic-Segmentation
```

Create a virtual environment:

```
python -m venv venv
```

Activate the virtual environment:

Windows

```
venv\Scripts\activate
```

Linux / macOS

```
source venv/bin/activate
```

Install backend dependencies:

```
pip install -r requirements.txt
```

7.3 Environment Configuration

Create a .env file using the provided example:

```
cp .env.example .env
```

This file is used to configure environment-specific parameters if required.

7.4 Running the Backend Pipeline

Run the backend processing pipeline using the pipeline controller:

```
python src/pipeline_controller.py
```

Processing Steps

- Audio preprocessing
 - Speech-to-text transcription
 - Transcript normalization
 - Topic segmentation
 - Keyword extraction
 - Sentiment analysis
 - Transcription quality evaluation
-

7.5 Output Generation

The generated outputs are stored in structured directories:

- Transcripts → Inference/transcripts/
- Segments → Inference/segments/

- Keywords → Included inside segment JSON files
- Quality Metrics → Word Error Rate (WER), Character Error Rate (CER), semantic similarity

7.6 Frontend Visualization (Optional)

To visualize results using the frontend interface:

```
cd ui_app
```

```
npm install
```

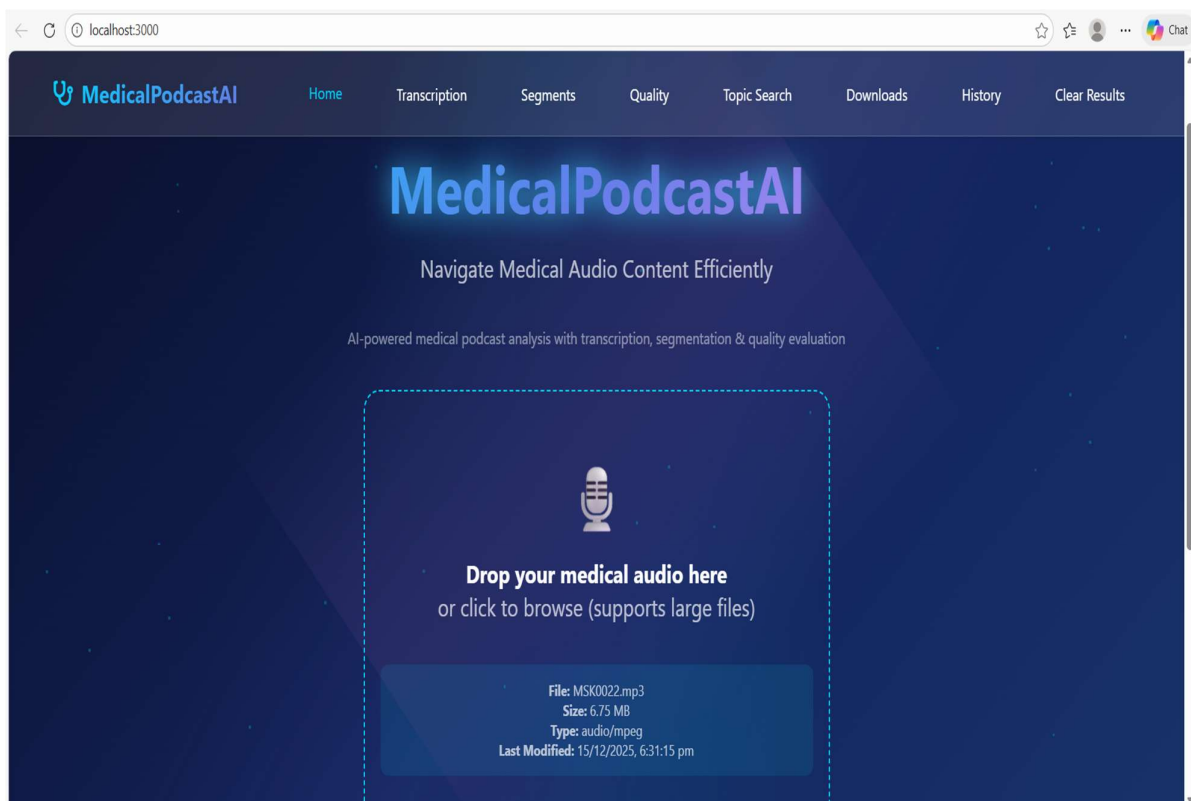
```
npm start
```

The application can be accessed at:

<http://localhost:3000>

7.7 User Interface Preview

Figure -7.1: UI Landing Page of the Automated Medical Podcast Transcription and Topic Segmentation System



7.8 Testing and Validation

Backend modules are validated using Pytest:

pytest -v

This ensures correctness of segmentation, keyword extraction, sentiment analysis, and quality evaluation.

7.9 Notes on Usage

- Audio files are processed offline and are not permanently stored
 - Large inference outputs are excluded from version control
 - The system is intended strictly for educational and analytical use
-

8. Technology Stack

The proposed system is developed using a layered technology stack to support efficient audio processing, natural language analysis, and interactive visualization. Figure 7.1 illustrates the overall technology stack used in the implementation.

The frontend layer is built using React.js, HTML, CSS, and JavaScript to provide a responsive and interactive user interface. Data visualization is achieved using Chart.js, Plotly, and Matplotlib. Communication with the backend is handled through REST APIs.

The API layer is implemented using Flask, which manages request handling and data exchange between the frontend and backend components.

The backend processing layer is developed using Python 3.9+ and is responsible for audio preprocessing and transcription. Whisper or Faster-Whisper models are used for speech-to-text conversion, while Librosa, PyDub, and FFmpeg are used for audio processing tasks.

The NLP and Machine Learning layer performs topic segmentation, keyword extraction, and sentiment analysis using HuggingFace Transformers, Sentence Transformers, NLTK, SpaCy, KeyBERT, YAKE, and RAKE.

The storage layer manages structured and semi-structured data using JSON and CSV formats, with optional support for SQLite and FAISS for persistent storage and similarity search.

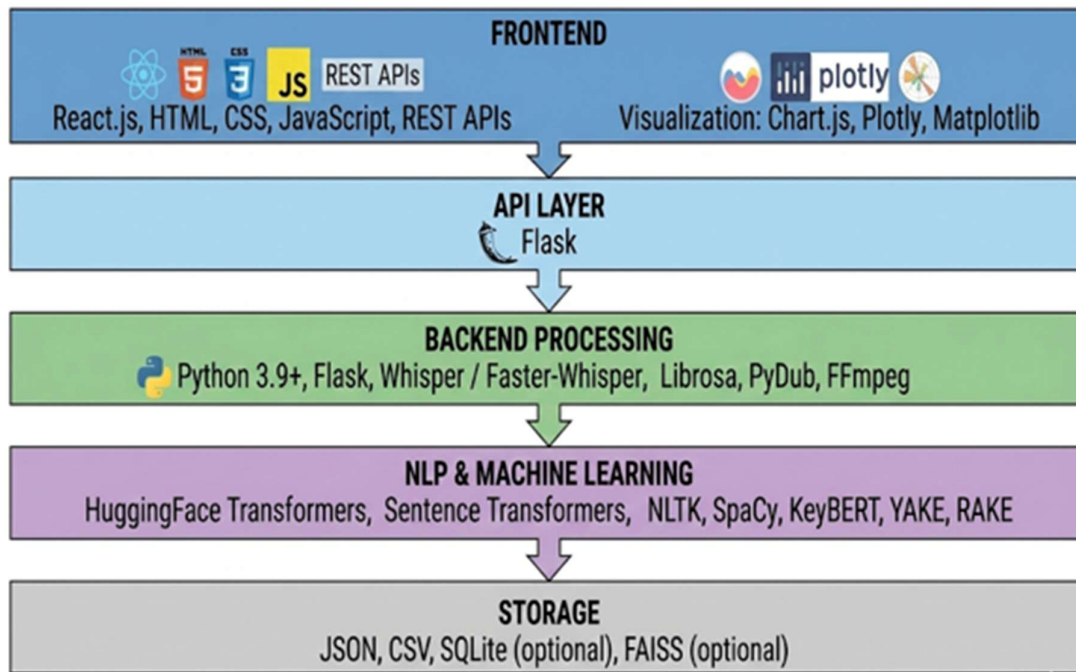


Figure 8.1 Layered Technology Stack used for Automated Medical Podcast Transcription and Topic Segmentation

9. Project Structure

9.1 Backend Structure

File	Description
transcription.py	Converts audio into text and generates timestamps
segmentation.py	Detects topic boundaries using embeddings
evaluation_summary.py	Computes WER, CER, accuracy, similarity
sentiment_analysis.py	Performs medical-aware sentiment analysis
keyword_cloud.py	Extracts and counts medical keywords
logger.py	Tracks pipeline execution and errors

9.2 API Layer

The API layer is implemented using Flask. The backend pipeline controller manages request handling, triggers the audio processing and analysis modules, and communicates processed results to the frontend through RESTful APIs.

9.3 Frontend Structure

The frontend provides dashboards and visual analytics for transcripts, topics, sentiment, and quality metrics. Timestamp-based navigation allows efficient access to specific sections.

9.4 Data and Output Folders

Separate folders are maintained for clean reference transcripts, generated transcripts, and output files such as JSON and CSV. This structure ensures organized data management.

10. Milestone-wise Implementation

The project was implemented in phases starting with audio preprocessing and transcription, followed by topic segmentation and keyword extraction. Sentiment analysis and quality evaluation were implemented next, and frontend integration and visualization were completed later, followed by documentation and final submission.

11. Data and Privacy Considerations

Audio files are processed temporarily and are not permanently stored. API keys are handled securely using environment variables. No personal user data is stored.

12. Challenges Faced

Some challenges included handling medical terminology accurately, managing background noise in audio, and tuning topic segmentation thresholds. Ensuring correct sentiment classification for medical discussions was also a challenge.

13. Limitations

The system supports only recorded audio files and does not handle real-time transcription. Accuracy may vary depending on audio quality and speaker clarity. The system does not provide clinical diagnosis.

14. Troubleshooting

Low transcription accuracy can be improved by enhancing audio quality. Topic segmentation issues can be resolved by adjusting similarity thresholds. Errors are tracked using the logging module for debugging.

15. Future Scope

Future enhancements include Medical Named Entity Recognition, speaker diarization, multilingual transcription support, semantic search, and automatic summarization of topics.

16. Intended Users

Medical students, healthcare professionals, medical researchers, and medical educators.

17. License

This project is licensed under the **MIT License**.

18. References

1. OpenAI Whisper Documentation
 2. HuggingFace Transformers Documentation
 3. Sentence Transformers Research Papers
 4. Librosa Audio Processing Library Documentation
 5. Research studies on ASR and topic segmentation
-